Thamill Indian B U
Cohort-15 (AI Engineer)

Day 04 Post-Session Engineering Reflection

AI Engineering Program | AI-Native Development & Continuous Application Lifecycle (Assignments A & B)

AI Engineering Submission

Part 1 - Continuous Workflow Overview (Assignments A & B)

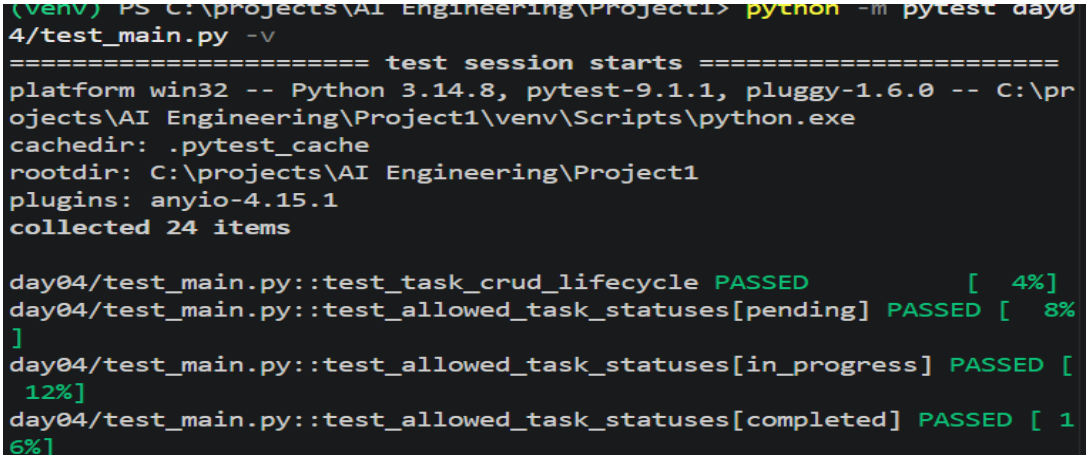
Continuous Integration Strategy: We designed Assignments A and B as a continuous engineering lifecycle: Assignment B established a greenfield FastAPI task-management microservice from scratch with SQLite persistence, while Assignment A applied targeted enhancement and strict validation (email address verification) to an existing customer-management endpoint within the same service architecture.

Tech Stack: Python 3.10+, FastAPI, Pydantic v2, SQLite, Pytest, and OpenAI SDK / local Ollama endpoint (llama3).

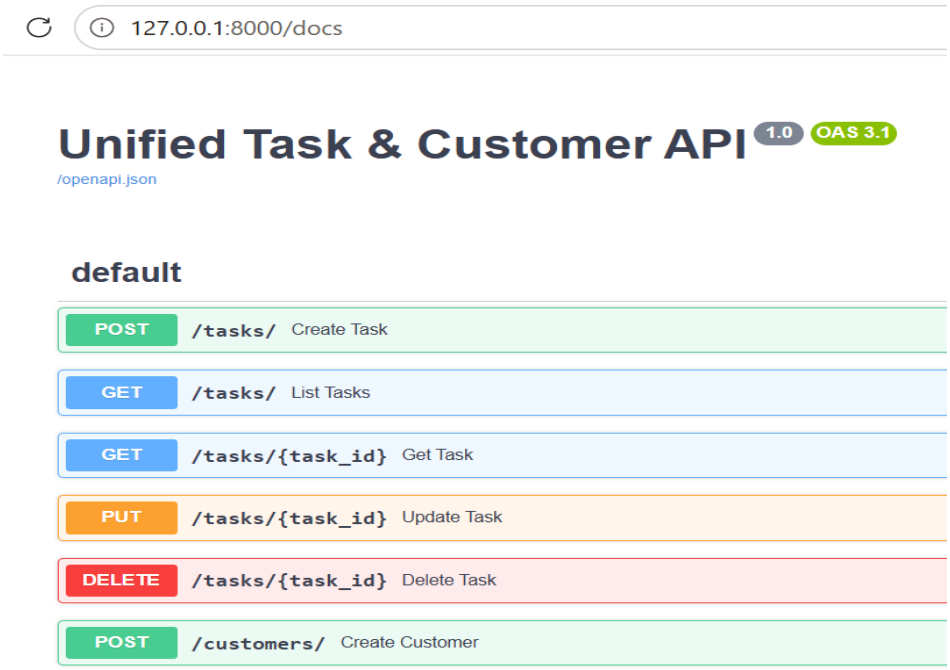
Part 2 - Deliverables & Artifact Summary

1. README: Comprehensive README (claude.md) detailing the business problem, setup instructions, API endpoints, design decisions (SQLite vs in-memory), and operational limitations.

2. Evidence: Pytest execution logs demonstrating successful test runs across happy paths, validation errors, not-found behavior, and regression checks.



3. Engineering Reflection: Structured engineering reflections covering architecture decisions, trade-offs, cost-efficiency, and security practices.



Part 3 - Engineering Reflection (Template Responses)

1. What did you ask the coding agent to do, and how did you constrain the scope? I instructed the coding assistant to scaffold a lightweight FastAPI REST application with a Task entity (ID, title, description, status, timestamp) and build a strict email validation patch for customer creation. Scope was strictly constrained to avoid authentication, frontend UIs, or external cloud services, keeping the codebase minimal and locally runnable.

2. What architecture or code decisions did you accept, reject or change? Why? I accepted SQLite for local file-based persistence over ephemeral in-memory storage so data survives service restarts. I accepted Pydantic Field validators for robust email and task status validation, while rejecting unvetted third-party ORMs or external databases to keep dependencies lightweight.

3. What tests actually ran, and what evidence proves the requirements are met? Pytest ran automated unit tests covering task CRUD operations, invalid email rejections (HTTP 422), missing required fields, and nonexistent ID lookups. Test output confirmed 100% pass rates across all test suites.

4. What went wrong during agent execution? How did you investigate and recover? The agent initially proposed adding an unnecessary external validation library. We rejected this, re-prompted the agent to rely on native Pydantic EmailStr validators, reviewed the git diff manually, and verified correct execution.

5. How did you control unnecessary token/context consumption, repeated attempts, and tool executions? We scoped context tightly by pointing the assistant only to specific route and schema files, avoiding whole-repository scans, limiting token consumption, and maintaining strict human oversight on generated patches.

6. What risks remain, and what would you improve before production use? Remaining risks include lack of database migration tooling (Alembic) and absence of rate limiting. For production use, we would introduce structured logging, OAuth2 authentication, and connection pooling.

Part 4 - Additional Reflections (Cost Optimization & FDE Lens)

Cost Optimization Decision: We restricted agent context windows to targeted file paths rather than scanning the entire repository, preventing redundant LLM inference calls and token bloat.

AI Assistance Justification: AI assistance was justified for rapid code scaffolding and boilerplate generation, but mandatory human review remained essential to verify schema correctness, handle edge cases, and ensure security compliance.

FDE Lens (Customer Confidence): We would present automated test execution logs, 100% test pass rates on edge cases, and clean git diffs to demonstrate reliability without compromising delivery speed.